

ADMINISTRACION DE ESENCIAS

Practica semana 10

OPTIMIZACIÓN DE BASE DE DATOS: GRADOSTITULOS - SEPARACIÓN DE ÍNDICES

1 EL DESAFÍO Y OBJETIVO

Mejorar el rendimiento de la base de datos "GradosTitulos" separando los índices no agrupados.

1 SOLUCIÓN (SCRIPT)

- Mejorar el rendimiento de la base de datos "GradosTitulos" con `footentolr`.
- Mejorar el rendimiento de la base de datos "GradosTitulos" separando los índices no agrupados.

2 IMPLEMENTACIÓN DEL SCRIPT T-SQL

```
CREATE DATABASE GradosTitulos
USE master;
GO

-- Crear un nuevo Filegroup para índices
ALTER DATABASE GradosTitulos
ADD FILEGROUP IdxGroup;
GO

-- Agregar archivo físico al Filegroup
ALTER DATABASE GradosTitulos
ADD FILE
(
    NAME = 'IdxGroup_Data',
    FILENAME = 'C:\SQLData\IdxGroup_Data.ndf',
    SIZE = 50MB,
    FILEGROWTH = 10MB
)
TO FILEGROUP IdxGroup;
GO

-- Ejemplo de creación de índice no agrupado
CREATE NONCLUSTERED INDEX IX_Persona_DNI
ON Persona(DNI)
ON IdxGroup;
```

3 ¿POR QUÉ USAR UN FILEGROUP SECUNDARIO (IdxGroup)?

✓ REDUCIR CONTENCIÓN



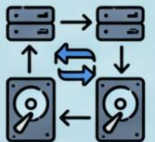
✓ MEJORAR RENDIMIENTO



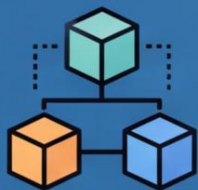
✓ FACILITAR MANTENIMIENTO Y RESPALDO



✓ DISTRIBUIR CARGA DE I/O



4 RESUMEN DE BUENAS PRÁCTICAS DE BASE DE DATOS



SEPARACIÓN LÓGICA Y FÍSICA

Organización y administración mejoradas.



OPTIMIZACIÓN DE RENDIMIENTO

Acelera búsquedas y filtros.



ESCALABILIDAD

Facilita el crecimiento futuro de índices.

Problema 2: Monitoreo Preventivo de Espacio en Base de Datos

Base de Datos GradosTitulos

1. ENUNCIADO Y OBJETIVO



DESARROLLAR UNA CONSULTA para visualizar el espacio asignado, utilizado y libre de todos los archivos físicos y grupos de archivos (Filegroups) de la base de datos.

Asignado



Utilizado



Libre



2. SCRIPT DE CONSULTA

```
USE GradosTitulos;
GO
SELECT
    fg.name AS Filegroup,
    df.name AS Archivo_Logico,
    df.physical_name AS Archivo_Fisico,
    df.size * 8 / 1024 AS Tamaño_MB,
    FILEPROPERTY(df.name, 'SpaceUsed') * 8 / 1024 AS Usado_MB,
    (df.size - FILEPROPERTY(df.name, 'SpaceUsed')) * 8 / 1024 AS Libre_MB
FROM sys.database_files df
LEFT JOIN sys.filegroups fg
    ON df.data_space_id = fg.data_space_id
ORDER BY fg.name;
```

3. JUSTIFICACIÓN Y EXPLICACIÓN CLAVE



1

Fuentes de Datos

Uso de Vistas de Catálogo oficiales (sys.database_files, sys.filegroups) para información confiable y actualizada.



2

Acción Preventiva

Detección de problemas de capacidad antes de que afecten el sistema.



3

Conversión de Unidades

Conversión de páginas (8 KB) a Megabytes (MB) para interpretación administrativa (size * 8 / 1024).



4

Organización

Resultados ordenados por Filegroup (ORDER BY) para facilitar el análisis.

EJERCICIO 3: CONFIGURAR PAGE_VERIFY CHECKSUM EN GRADOSTITULOS

PROTECCIÓN DE INTEGRIDAD DE DATOS CONTRA FALLOS DE HARDWARE, DISCO O SO

1 CONSULTAR CONFIGURACIÓN ACTUAL (Pre-Cambio)

```
SELECT
    name AS BaseDatos,
    page_verify_option_desc AS VerificacionPagina
FROM sys.databases
WHERE name = 'GradosTitulos';
```



Configuración Actual

2 EJECUTAR SCRIPT DE CAMBIO

```
USE [master]
GO
ALTER DATABASE [GradosTitulos]
    SET PAGE_VERIFY CHECKSUM WITH ROLLBACK IMMEDIATE;
GO
```



3 VALIDAR CAMBIO (Post-Cambio)

```
SELECT
    name AS BaseDatos,
    page_verify_option_desc AS VerificacionPagina
FROM sys.databases
WHERE name = 'GradosTitulos';
GO
```



Configuración CHECKSUM Aplicada

POR QUÉ CHECKSUM ES ESENCIAL (Justificación)

- La opción **CHECKSUM** genera y almacena una suma de verificación para cada página al escribir en disco.
- Al leer, SQL Server compara el valor almacenado con uno recién calculado.
- Si no coinciden, detecta corrupción física inmediata.

EXPLICACIÓN Y BENEFICIOS ADICIONALES



PROTECCIÓN DE INTEGRIDAD: Ayuda a detectar daños físicos antes de que afecten gravemente la información.



CONFIGURACIÓN RECOMENDADA POR MICROSOFT: Para entornos de producción por alta detección de errores.



VERIFICACIÓN COMPLETA: Se consulta antes y después para validar el cambio aplicado.



MANTENIMIENTO PREVENTIVO: Permite identificar problemas de hardware o almacenamiento de forma temprana.

EJERCICIO 5) EJERCICIO) ESTRATEGIA DE RESPALDO DIFERENCIAL (DIFFERENTIAL BACKUP)

OBJETIVO: Respaldo diferencial de GradosTitulos

1. EL PROBLEMA & LA NECESIDAD


Full Backup Diarios
(Lento, Gran Espacio)

VS


Respaldo Diferencial
(Rápido, Menos Espacio)

2. JUSTIFICACIÓN: VENTAJAS PRINCIPALES

• **TAMAÑO REDUCIDO**
Garantiza o reducido



• **MENOR TIEMPO DE EJECUCIÓN**



• **AHORRO DE RECURSOS DEL SERVIDOR**



• **RESTAURACIÓN SIMPLIFICADA**



3. EL SCRIPT SQL

```
SELECT
    name AS BaseDatos,
    recovery_model_desc AS ModeloRecuperacion
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Diferencial.bak'
WITH DIFFERENTIAL,
    INIT,
    STATS = 10;
GO
```

DATABASE NAME

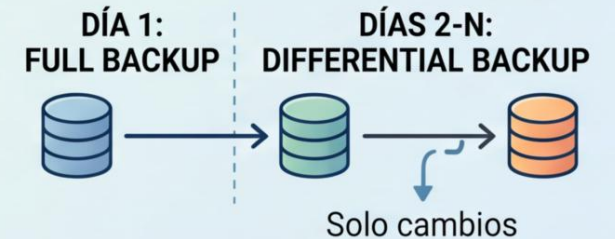
TARGET FILE PATH

DIFFERENTIAL TYPE

INIT OVERWRITE

4. ESTRATEGIA EXPLICADA

(A) Estrategia de respaldo escalonada



(B) Uso de INIT y Validación



1. INIT (Sobrescribir file)
Garantiza sobrescritura, evita acumulación innecesaria



2. Validación posterior RESTORE HEADERONLY
Verifica el respaldo creado correctamente



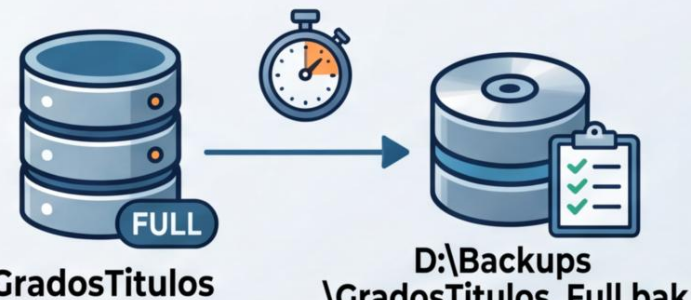
EJERCICIO 6: RESPALDO DEL REGISTRO DE TRANSACCIONES

```
ALTER DATABASE GradosTitulos
SET RECOVERY FULL;
GO
BACKUP DATABASE GradosTitulos
TO DISK =
'D:\Backups\GradosTitulos_Full.bak'
WITH INIT,
STATS = 10;
GO
BACKUP LOG GradosTitulos
TO DISK =
'D:\Backups\GradosTitulos_Log.trn'
WITH INIT,
STATS = 10;
GO
```

EXECUTE

EXECUTE

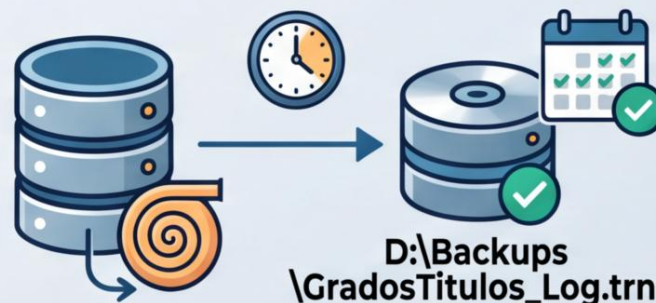
2



1. CONFIGURACIÓN Y RESPALDO COMPLETO INICIAL:

Establece el modelo de recuperación y genera el primer punto de referencia.

1



2. RESPALDO DEL LOG (TRANSACCIONES POSTERIORES):
Captura los cambios desde el último respaldo completo.

3

4

JUSTIFICACIÓN (BENEFICIOS)



Punto en el tiempo:
Recuperación hasta un momento exacto.



Pérdida mínima de datos:
Maximiza la protección.



Liberación de espacio:
Trunca el archivo .ldf para reutilizar espacio.



Protección ante fallos.

5

EXPLICACIÓN TÉCNICA



Uso del Modelo FULL:
Estrategias avanzadas de recuperación.



Respaldo completo previo:
Requerido para iniciar la cadena de recuperación.



Separación de archivos:
FULL (.bak) y LOG (.trn) en archivos independientes.

CREACIÓN Y GESTIÓN DE ROLES DE BASE DE DATOS PERSONALIZADOS

EJERCICIO 7: RolSecretariaAcademica



CONTEXTO Y OBJETIVO

Crear un rol personalizado 'RolSecretariaAcademica' en la base de datos 'GradosTitulos' para administrar permisos de forma segura y eficiente.



Base de Datos
GradosTitulos

GradosTitulos



Rol a Crear:
RolSecretariaAcademica



Base de Tabla **GradosTitulos**

```
-- Creación del Rol
CREATE ROLE RolSecretariaAcademica;

-- Permisos de Consulta en Esquemas
GRANT SELECT ON SCHEMA::Academico TO RolSecretariaAcademica;
GRANT SELECT ON SCHEMA::Tramite TO RolSecretariaAcademica;
GO
```

ASIGNACIÓN DE PERMISOS (Principio de Mínimo Privilegio)

Esquema ACADEMICO		Esquema TRAMITE	
SELECT (Consulta)	✓ AUTORIZADO (Consulta) en todas las tablas	SELECT (Consulta)	✓ AUTORIZADO
SELECT (Consulta) en todas las tablas	✗ RESTRINGIDO INSERT UPDATE DELETE	INSERT (Inserción)	SOLO Tabla Tramite.Tramite
		UPDATE	✗ RESTRINGIDO
		DELETE	✗ RESTRINGIDO

Integridad de Datos (lock icon)

NO se conceden permisos de modificación (UPDATE) ni eliminación (DELETE) para proteger la integridad de los datos

Acceso ses only hasia barera for lectura

SCRIPT DE SQL (Para referencia)

```
-- Creación del Rol
CREATE ROLE RolSecretariaAcademica;

-- Permisos de Consulta en Esquemas
GRANT SELECT ON SCHEMA::Academico TO RolSecretariaAcademica;
GRANT SELECT ON SCHEMA::Tramite TO RolSecretariaAcademica;

-- Permiso de Inserción Especifico
GRANT INSERT ON OBJECT::Tramite.Tramite TO RolSecretariaAcademica;
GO
```

Nota: Script proporcionado es solo referencial y no coincide con la imagen de ejemplo que no pertenece a este ejercicio

JUSTIFICACIÓN Y EXPLICACIÓN



Uso de Roles Personalizados

Permite administrar permisos de forma centralizada y reutilizable.



Principio de Mínimo Privilegio

Los usuarios reciben únicamente los permisos necesarios.



Asignación por Esquema

Simplifica la administración cuando existen múltiples tablas relacionadas.



Separación de Responsabilidades

Las operaciones de consulta y registro se controlan independientemente.

EJERCICIO 8: Restringir Explícitamente Acceso de Lectura a Tramite.Pago

Restringir al usuario **OperadorTramite** acceso a **Tramite.Pago**, asegurando que la denegación **PREVALEZCA** sobre otros roles/permisos.

Script de Implementación (Basado en la imagen)

```
1  USE GradosTitulos;
2  GO
3
4  -- Denegar lectura sobre la tabla Tramite.Pago
5  DENY SELECT
6  ON Tramite.Pago
7  TO OperadorTramite;
8  GO
9
10 -- Verificar permisos aplicados
11 SELECT
12     dp.state_desc,
13     dp.permission_name,
14     OBJECT_NAME(dp.major_id) AS Objeto
15 FROM sys.database_permissions dp
16 WHERE dp.grantee_principal_id =
17        USER_ID('OperadorTramite');
```

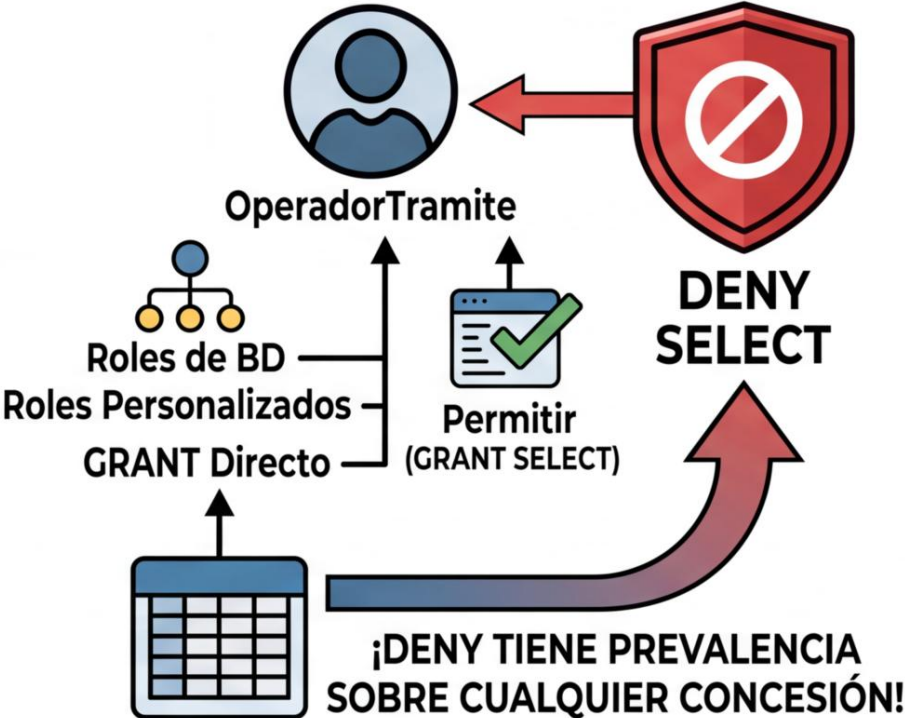
Acción Principal:
BLOQUEO EXPLÍCITO

Pago ID	Monto	Fecha
Pago		04/02/...
Pago		11/02/...
Pago		12/01/...
Pago		01/02/...
Pago		01/01/...
Pago		01/01/...



ACCESO RESTRINGIDO

Diagrama de Prevalencia de Permisos



1. JUSTIFICACIÓN: BLOQUEO TOTAL

La instrucción DENY bloquea explícitamente y tiene prioridad absoluta.



2. PROTECCIÓN DE INFO. CRÍTICA

Datos financieros (Pago) requieren seguridad adicional.



3. CONTROL GRANULAR

Restricción específica sin afectar a otros roles o tablas.



4. VERIFICACIÓN Y AUDITORÍA

Consulta SQL incluida para validar permisos.

SOLUCIÓN VERIFICADA



EJERCICIO 9: SEGURIDAD DE DATOS - PERMISOS A NIVEL DE COLUMNA



**Enunciado y Contexto



Otorgar permisos **SELECT** únicamente sobre columnas específicas al usuario **PracticanteSistemas**



**Columnas Solicitadas (Permitidas)

IdPersona
Nombres
ApellidoPaterno
Correo



**Otras Columnas (Restringidas - Sensibles)

DNI
Teléfono
Dirección
Fecha de nacimiento
Información administrativa

Tabla
Academico.Persona



IMPLEMENTADO CORRECTAMENTE



```
USE GradosTitulos;
GO

-- Otorgar permisos SELECT únicamente sobre columnas específicas
GRANT SELECT
(
    IdPersona,
    Nombres,
    ApellidoPaterno,
    Correo
)
ON OBJECT::Academico.Persona
TO PracticanteSistemas;
GO

-- Verificar permisos asignados
SELECT
    dp.permission_name,
    dp.state_desc,
    OBJECT_NAME(dp.major_id) AS Objeto
FROM sys.database_permissions dp
WHERE dp.grantee_principal_id =
    USER_ID('PracticanteSistemas');
GO
```

Otorgar permisos SELECT únicamente sobre columnas específicas al usuario **PracticanteSistemas**

ON OBJECT::Academico.Persona
TO PracticanteSistemas;



Protección de datos sensibles

Información personal y confidencial a comunidad a. restringida.

Cumplimiento normativo

Ayuda a cumplir políticas institucionales.

Reducción de riesgos

Disminuye la exposición

**Justificación: Necesidad del Negocio

Los permisos a nivel de columna permiten un control preciso. El usuario requiere datos de contacto básicos.

Otras columnas contienen información sensible (DNI, Teléfono).

Se limita el acceso para el desempeño de funciones.

**Explicación: Beneficios y Principios

1. **Aplicación del principio de mínimo privilegio**
Permisos indispensables para las actividades.
2. **Protección de datos sensibles**
Información personal y confidencial restringida.
3. **Seguridad granular**
Permisos asignados directamente a columnas.
4. **Cumplimiento normativo**
Ayuda a cumplir políticas institucionales.
5. **Reducción de riesgos**
Disminuye la exposición accidental.

SEGURIDAD DE BASE DE DATOS: PERMISO EXECUTE Y PROCEDIMIENTOS ALMACENADOS

Enunciado del Requerimiento



Conceder al usuario 'OperadorTramite' permisos **EXCLUSIVOS** para ejecutar el procedimiento almacenado 'Tramite.SP_RegistrarTramite'.

Objetivo: Permitir registrar nuevas solicitudes.

Restricción: SIN acceso directo de escritura sobre las tablas involucradas.

Script SQL de Implementación y Verificación

```
USE GradosTitulos;
GO

-- Otorgar permiso de ejecución
GRANT EXECUTE
ON OBJECT::Tramite.SP_RegistrarTramite
TO OperadorTramite;
GO

-- Verificar permisos otorgados
SELECT
    dp.permission_name,
    dp.state_desc,
    OBJECT_NAME (dp.major_id) AS Objeto
FROM sys.database_permissions dp
WHERE dp.grantee_principal_id =
    USER_ID('OperadorTramite');
GO
```

Justificación del Permiso EXECUTE



Control centralizado de lógica de negocio.



Mayor seguridad sobre los datos.



Restricción de accesos directos a tablas críticas.



Validación de información antes de inserciones.



Facilidad para auditar operaciones.



EJERCICIO 10: PERMISOS DE EJECUCIÓN (EXECUTE PERMISSION)

Enunciado

Conceder al usuario **"OperadorTramite"** permisos exclusivos para ejecutar el procedimiento almacenado **"Tramite.SP_RegistrarTramite"**, permitiéndole registrar nuevas solicitudes sin otorgarle acceso directo de escritura sobre las tablas involucradas.

Code Editor View

```
1  USE GradosTitulos;
2  GO
3
4  -- Otorgar permiso de ejecución sobre el procedimiento
5  GRANT EXECUTE
6  ON OBJECT::Tramite.SP_RegistrarTramite
7  TO OperadorTramite;
8  GO
9
10 -- Verificar permisos otorgados
11 SELECT
12     dp.permission_name,
13     dp.state_desc,
14     OBJECT_NAME(dp.major_id) AS Objeto
15 FROM sys.database_permissions dp
16 WHERE dp.grantee_principal_id =
17     USER_ID('OperadorTramite');
18 GO
```

EXECUCION VISUALIZED



VERIFICACIÓN DE PERMISOS (RESULTADO)

	permission_name	state_desc	Objeto
1	EXECUTE	GRANT	SP_RegistrarTramite
2	OperadorTramite	GRANT	OperadorTramite

JUSTIFICACIÓN Y EXPLICACIÓN



Control centralizado

Conceder al usuario permisos para ejecutar el procedimiento.



Seguridad basada en ejecución

Seguridad basada en ejecución permitiéndole registrar solicitudes.



Restricción de accesos directos

Restricción de accesos directos como eoa tablas involucradas.



Menor superficie de ataque

Menor superficie de ataque y beneficie diseño.



Mantenimiento simplificado

Mantenimiento simplificado compraron el nxuramento.

EJERCICIO 11) Monitoreo de Consumo de CPU de Consultas T-SQL en la Base de Datos GradosTitulos



Enunciado:

Monitoreo del consumo de CPU de Consultas T-SQL en la Base de Datos GradosTitulos (mv. en mvtatric y tadicalhs. La, monitoreo de fiail consumo de CPU de Consultas T-SQL en la Base de Datos GradosTitulos.

1 Script

```
SELECT TOP 5
    qs.total_worker_time / 1000 AS CPU_Total_ms,
    qs.execution_count AS Veces_Ejecutada,
    (qs.total_worker_time / qs.execution_count / 1000 AS CPU_Promedio_ms,
    qs.total_elapsed_time / 1000 AS Tiempo_Total_ms,
    DB_NAME(st.dbid) AS BaseDatos,
    SUBSTRING(st.text,
        (qs.statement_start_offset / 2) + 1,
        (
            CASE qs.statement_end_offset
            WHEN -1 THEN DATALENGTH(st.text)
            ELSE qs.statement_end_offset
            END
            - qs.statement_start_offset
        ) / 2
    ) + 1
    ) AS ConsultaSQL
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE DB_NAME(st.dbid) = 'GradosTitulos'
ORDER BY qs.total_worker_time DESC;
GO
```

2 Explicación

- 1 **Uso de DMVs oficiales:** Confiabilidad.
- 2 **Análisis basado en métricas reales:** Datos vs suposiciones.
- 3 **Priorización de problemas:** Top 5 consultas costosas.
- 4 **Monitoreo proactivo:** Detección de ineficiencia.
- 5 **Diagnóstico de rendimiento:** Identificación de cuellos de botella.

3 Justificación

La consulta utiliza la DMV: (**sys.dm_exec_query_stats**) para obtener estadísticas acumuladas de ejecución de consultas almacenadas en caché.

Los indicadores más importantes analizados son:

- **total_worker_time** → tiempo total de CPU consumido.
- **execution_count** → cantidad de ejecuciones.
- **total_elapsed_time** → tiempo total de ejecución.
- **sql_handle** → identificador de la consulta almacenada.

La combinación con:

- **sys.dm_exec_sql_text** → para obtener el texto de la consulta.

EJERCICIO 12: MONITOREO DE ÍNDICES NO AGRUPADOS

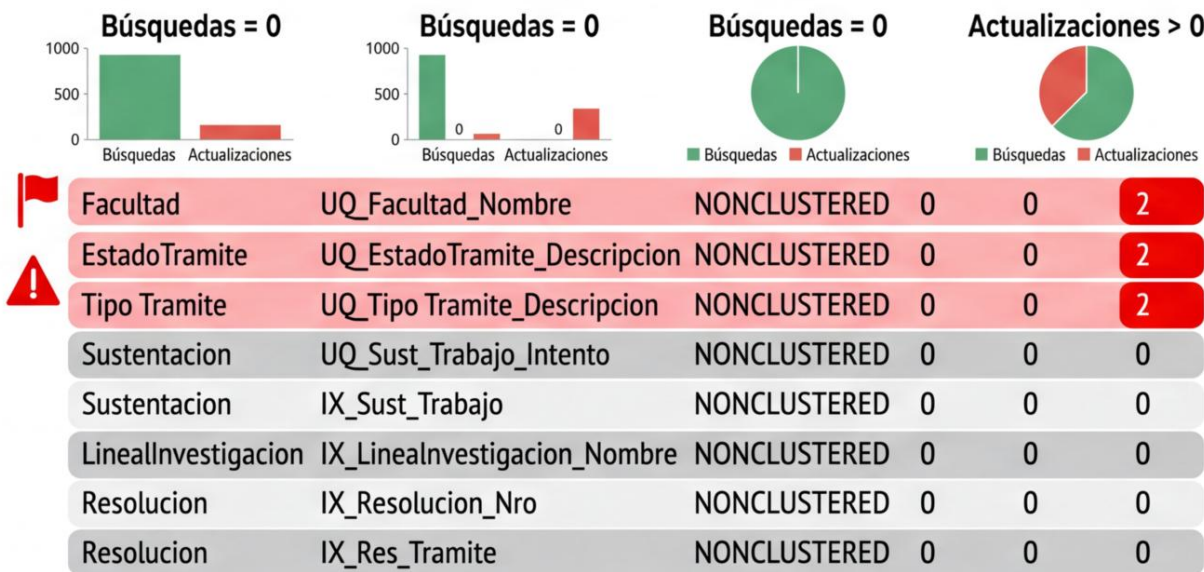
VERIFICACIÓN DE USO REAL DE ÍNDICES (Base de Datos GradosTitulos)



JUSTIFICACIÓN TÉCNICA (Uso de DMV)

- uses
sys.dm_db_index_usage_stats (DMV).
- metrics for:
 - user_seeks
 - Búsquedas eficientes)
 - user_scans
 - user_lookups
 - user_updates.
- **Atención:** Índice con Búsquedas = 0 y Actualizaciones > 0 es candidato para revisión.

```
1 SELECT
2     OBJECT_NAME(i.object_id) AS Tabla,
3     i.name AS Indice,
4     i.type_desc AS TipoIndice,
5     ISNULL(us.user_seeks,0) AS Busquedas,
6     ISNULL(us.user_scans,0) AS Escaneos,
7     ISNULL(us.user_lookups,0) AS Lookups,
8     ISNULL(us.user_updates,0) AS Actualizaciones,
9     us.last_user_seek AS UltimaBusqueda
10  FROM sys.indexes i
11  LEFT JOIN sys.dm_db_index_usage_stats us
12    ON i.object_id = us.object_id
13     AND i.index_id = us.index_id
14     AND us.database_id = DB_ID()
15  WHERE i.type_desc = 'NONCLUSTERED'
16         AND i.name IS NOT NULL
17  ORDER BY Busquedas ASC, Actualizaciones DESC;
18  GO
```



Fuente: sys.dm_db_index_usage_stats (GradosTitulos)



BENEFICIOS Y EXPLICACIÓN

- Monitoreo continuo del uso real.
- Identificación de índices innecesarios (recursos consumidos sin beneficio).
- Optimización del rendimiento (reduce carga de escritura).
- Métricas históricas (decisiones basadas en evidencia).
- Administración eficiente (equilibrio rendimiento-coste).